

二分与分治题目选讲1

北京大学 王欣灏



课前多叽歪

咱们不以赶进度为唯一目标，力求大家尽可能的理解吸收

所以大家有各式各样的问题请多多提问

各式各样的想法也请多多发言

碰到让你疑惑的地方，不妨说出来大家一起解决

总而言之，feel free to ask questions!

由于课程需要录制，在直播进行过程中提问和发言尽量在直播间两天窗口进行，不用害羞^v^

非直播时间（中午、晚上下课后）也可通过其他渠道向我直接提问
(qq:1879570236)



书的复制 洛谷1281

问题描述：

现在要把M本有顺序的书分给K个人复制（抄写），每一个人的抄写速度都一样，一本书不允许给两个（或以上）的人抄写，分给每一个人的书，必须是连续的，比如不能把第一、第三、第四本数给同一个人抄写。

现在请你设计一种方案，使得复制时间最短。复制时间为抄写页数最多的人用去的时间。

输入格式：

第一行两个整数M、K；（ $K \leq M \leq 100$ ）

第二行M个整数，第i个整数表示第i本书的页数。

输出格式：

共K行，每行两个正整数，第i行表示第i个人抄写的书的起始编号和终止编号。K行的起始编号应该从小到大排列，**如果有多解，则尽可能让前面的人少抄写。**



二分答案，将最优解转换为判定性问题。

先设定一个猜测答案的范围，然后再这个范围内，通过二分查找一个答案，判定该答案是否符合要求,直到找到一个最小值。

```
//初始化数据
for(i=1;i<=m;i++)
{
    scanf("%d",&a[i]);
    Right+=a[i];           //上界
    if(a[i]>Left)Left=a[i]; //下界
}

//二分讨论答案
while(Left<=Right)//Left初值为答案的下界，Right初值为答案的上界
{
    Mid=(Left+Right)/2;
    if(Ok(Mid))Right=Mid-1; //Ok() 函数用于判断该解是否符合要求
    else Left=Mid+1;
}
ans=Left;           //结果一定是Left所表示的数字
```



```
bool Ok(int x)                                //验证x是否是一个可行的答案
{
    int i,cnt=0,tot=0;                          //cnt用于记录当前工作安排给了第几个人
    for(i=1;i<=m;i++)
        if(tot+a[i]<=x)tot+=a[i]; //tot记录当前这份工作的总量
        else
        {
            cnt++;                          //cnt记录当前已安排的人数
            if(cnt>k)return false;
            tot=a[i];
        }
    cnt++; //最后一份工作要安排一个人来完成
    if(cnt<=k)return true; //若完成工作所需人数不超过K个人，说明答案合理
    else return false;
}
```



题目要求"如果有多解，则尽可能让前面的人少抄写"，这怎么处理？

倒序安排每个人的工作，尽可能让后面的人多干工作！

```
End[k]=m;    //End[i]记录第i个人抄的最后一本书的编号
Start[1]=1;   //Start[i]记录第i个人抄的第一本书的编
p=k;         //p记录当前讨论到第几个人了，先从最后一个人讨论，所以p=k
tot=0;       //tot记录当前这个人抄写的书本数量
for(i=m;i>=1;i--)
    if(tot+a[i]<=ans)tot+=a[i];
    else
    {
        Start[p]=i+1;
        tot=a[i];
        p--;
        End[p]=i;
    }
```



奇怪的函数 洛谷2759

问题描述：使得 x^x 达到或超过n位数字的最小正整数x是多少？

输入数据：输入一个正整数n。

输出数据：输出使得 x^x 达到n位数字的最小正整数x。

输入样例：11

输出样例：10

数据规模： $n \leq 2\,000\,000\,000$

解法：枚举x，一个个试x，直到找到合适的为止

怎样枚举x？ 二分枚举答案



分析： 位数为n的数字，最小一个是？ 10^{n-1}

$$10^{n-1} = x^x$$

$$\log(10^{n-1}) = \log(x^x)$$

$$(n-1) * \log(10) = x * \log(x)$$

$$n = x * \log(x) / \log(10) + 1$$

$$n = x * \log(x) / \log(10) + 1$$

只需枚举x, 只要x满足
 $n \leq x * \log^{(x)} / \log^{(10)} + 1$
那么x就是满足要求的。

也可以这样表示：

$$\log_{10}(10^{n-1}) = \log_{10}(x^x)$$

$$(n-1) * \log_{10}(10) = x * \log_{10}(x)$$

$$n = x * \log_{10}(x) + 1$$

c++中log()表示自然对数
log10()表示10为底的对数
两者结果均是double型
使用对数需引用math包
`#include <cmath>`

二分枚举x




```
#include <cmath>
```

```
.....
```

```
cin>>n;
```

```
Left=0;    Right=250000000;    //初始化二分答案的上下界
```

```
while(Left<=Right)
```

```
{
```

```
    x=(Left+Right)/2;
```

```
    temp=floor(x*log10(x)+1); //ceil()表示向上取整
```

```
    if(temp>=n)Right=x-1; else Left=x+1;
```

```
}
```

```
cout<<Left; //或者cout<<Right+1;
```

```
.....
```

ceil(x) 作用是对实数x进行向上取整，ceil意思是天花板
floor(x) 作用是对实数x进行向下取整，floor意思是地板



小车问题 洛谷1258

问题描述：

甲、乙两人同时从A地出发要尽快同时赶到B地。出发时A地有一辆小车，可是这辆小车除了驾驶员外只能带一人。已知甲、乙两人的步行速度一样，且小于车的速度。问：怎样利用小车才能使两人尽快同时到达。

输入数据：

仅一行，三个数据分别表示AB两地的距离 s ，人的步行速度 a ，车的速度 b 。

输出数据：

两人同时到达B地需要的最短时间(保留2个小数位)。

输入样例：

120 5 25

输出样例：

9.6000

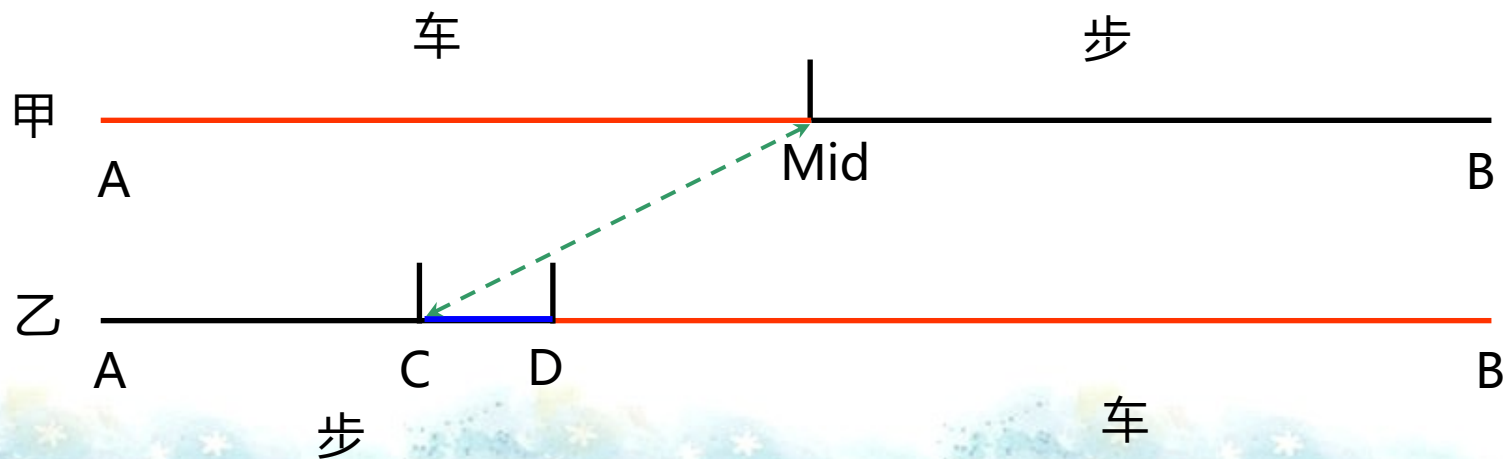


分析

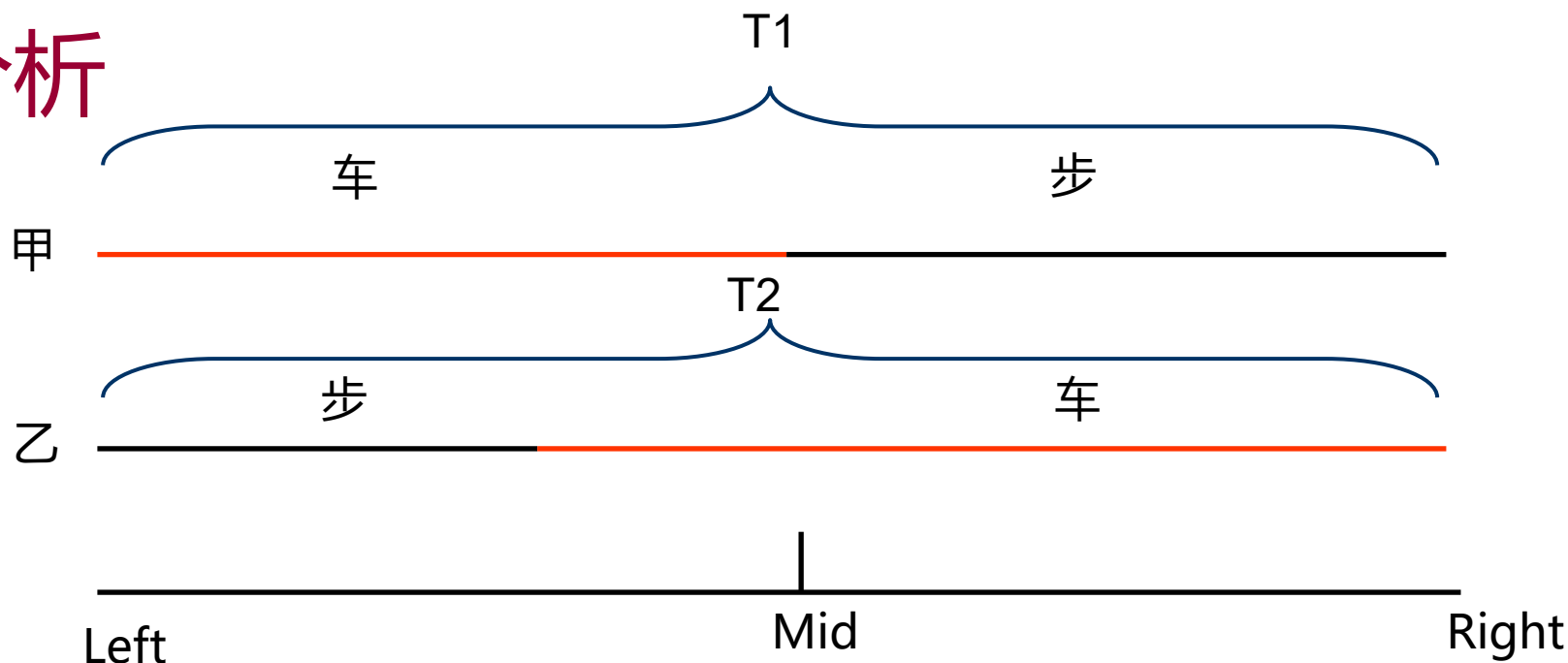
最佳方案为：

甲先乘车到达Mid处后下车步行，小车再回头接已走到C处的乙，在D处相遇后，乙再乘车赶往B，最后甲、乙一起到达B地。

这样问题就转换成了求Mid处的位置，使得甲乙到达终点B时花费的总时间相同。我们用二分法，不断尝试，直到满足同时到达的时间精度。



分析



设甲花费的总时间是 $T1$ ，乙花费的总时间是 $T2$ 。

乘车速度大于步行速度

甲一定是乘车到Mid，再步行到终点，乙步行的距离一定小于Mid

要求甲乙同时到达，也就是要使得 $T1 = T2$

如果 $T1 < T2$ ，则说明甲乘车的距离长了，Mid应该左移

如果 $T1 > T2$ ，则说明甲乘车的距离短了，Mid应该右移

因为 $Mid = (Left + Right) / 2$

要让Mid左移，Left保持不变，新的 $Right = Mid$

要让Mid右移，Right保持不变，新的 $Left = Mid$



分析

```
Left=0, Right=s;  
while(fabs(T1-T2)>精度要求)  
{  
    Mid = (Left+Right)/2;  
    求甲的时间T1, 乙的时间T2;  
    if (T1<T2) Right = Mid;  
    else Left = Mid;  
}
```

精度要求可以认为是一个很小的实数, 例如0.00001



关键代码

```
void BinaryAnswer( )
```

```
{
```

```
    double Left, Right, Mid, T1, T2, T3, D1, D2;
```

```
    Left = 0;           //二分答案的下界
```

```
    Right = S;          //二分答案的上界, S为题目给出的AB两地的距离
```

```
    T1 = S / car;        //时间初值
```

```
    T2 = S / walk;
```

```
    while(fabs(T1 - T2) > 0.0001)
```

//fabs(x)表示求实数x的绝对值

```
{
```

```
    Mid = (Left + Right) / 2;
```

//二分讨论甲下车的位置

```
    ///////////////求甲的时间T1, 乙的时间T2/////////////////
```

```
    T1 = Mid / car + (S - Mid) / walk;
```

//甲的总耗时

```
    D1 = (Mid / car) * walk;
```

//甲坐车时, 乙走的距离, 图中A到C

```
    T3 = (Mid - D1) / (car + walk);
```

//车返程接乙的时间, 图中乙从C走路到D的时间

```
    D2 = T3 * walk;
```

//甲下车到乙上车, 乙又走的路程, 图中C到D

```
    T2 = Mid / car + T3 + (S - D1 - D2) / car;
```

//乙的总耗时

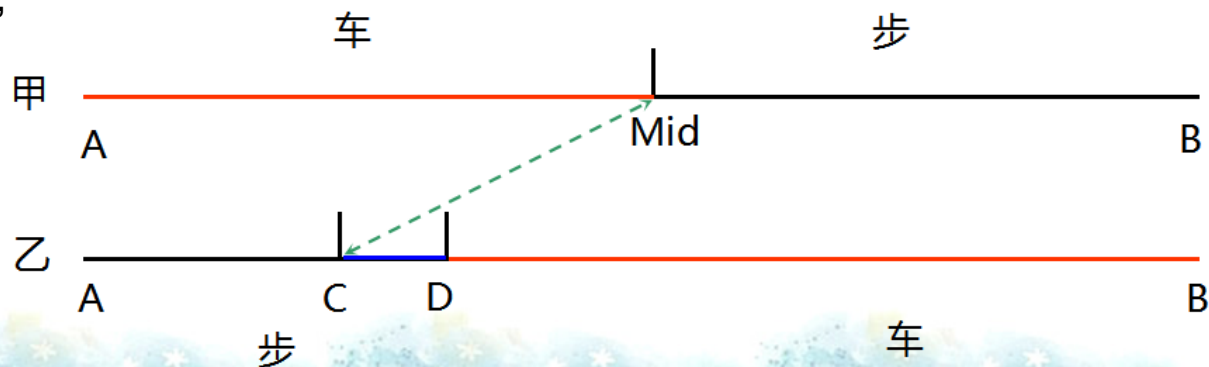
```
    ///////////////////////////////////////////////////////////////////
```

```
    if(T1 > T2) Left = Mid;
```

```
    else Right = Mid;
```

```
}
```

```
}
```



最近点对 洛谷1429

问题描述:

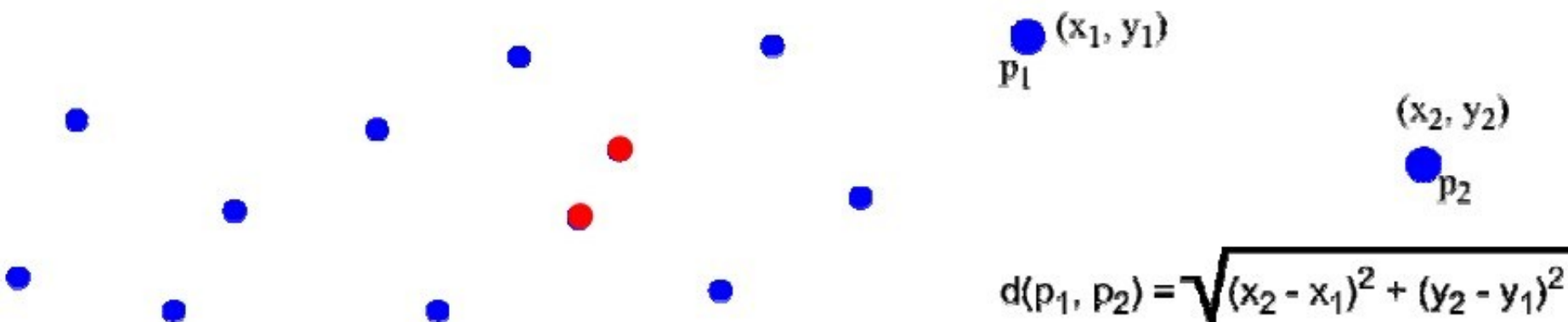
给定坐标平面上 n ($n \leq 60000$)个点,找出其中的一对点的距离,使得在这 n 个点的所有点对中,该距离为所有点对中最小的。



分析

【问题简述】 给定平面上 n 个点的坐标，找出其中欧几里德距离最近的两个点。

【方法1】 枚举算法。 需要枚举 $O(n^2)$ 个点对，每个距离的计算时间为 $O(1)$ ，故总的时间复杂度为 $O(n^2)$ 。



有没有更好的算法呢？



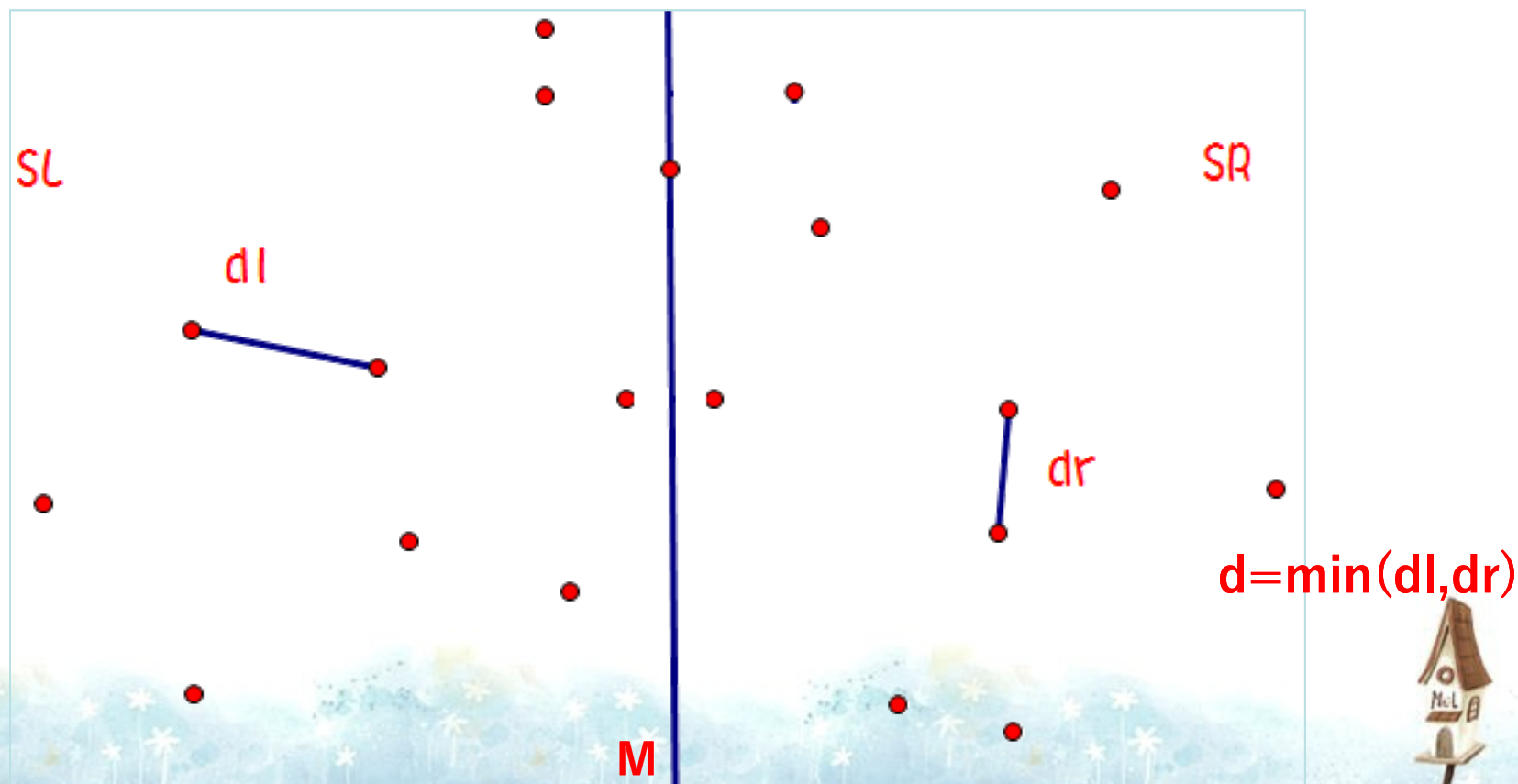
【算法分析】 已知一个点的集合S，求集合中欧几里得距离最小的一对点。

算法准备：对点集S进行排序，一般按点的X坐标从小到大排(这一步也称为“离散化”)。

算法过程：每次选取一条垂直x轴的直线M，将S拆分成左右两部分SL，SR,如下图。

M一般按排序后的点集S中的**编号位于中间的那一个点的x坐标来划分**，这样可以保证SL和SR中的点数目各约为 $n/2$ ，否则，算法的效率就很难保证了。

依次找到SL，SR中的最短距离 d_l ， d_r 。记录最短距离 $d = \min(d_l, d_r)$;

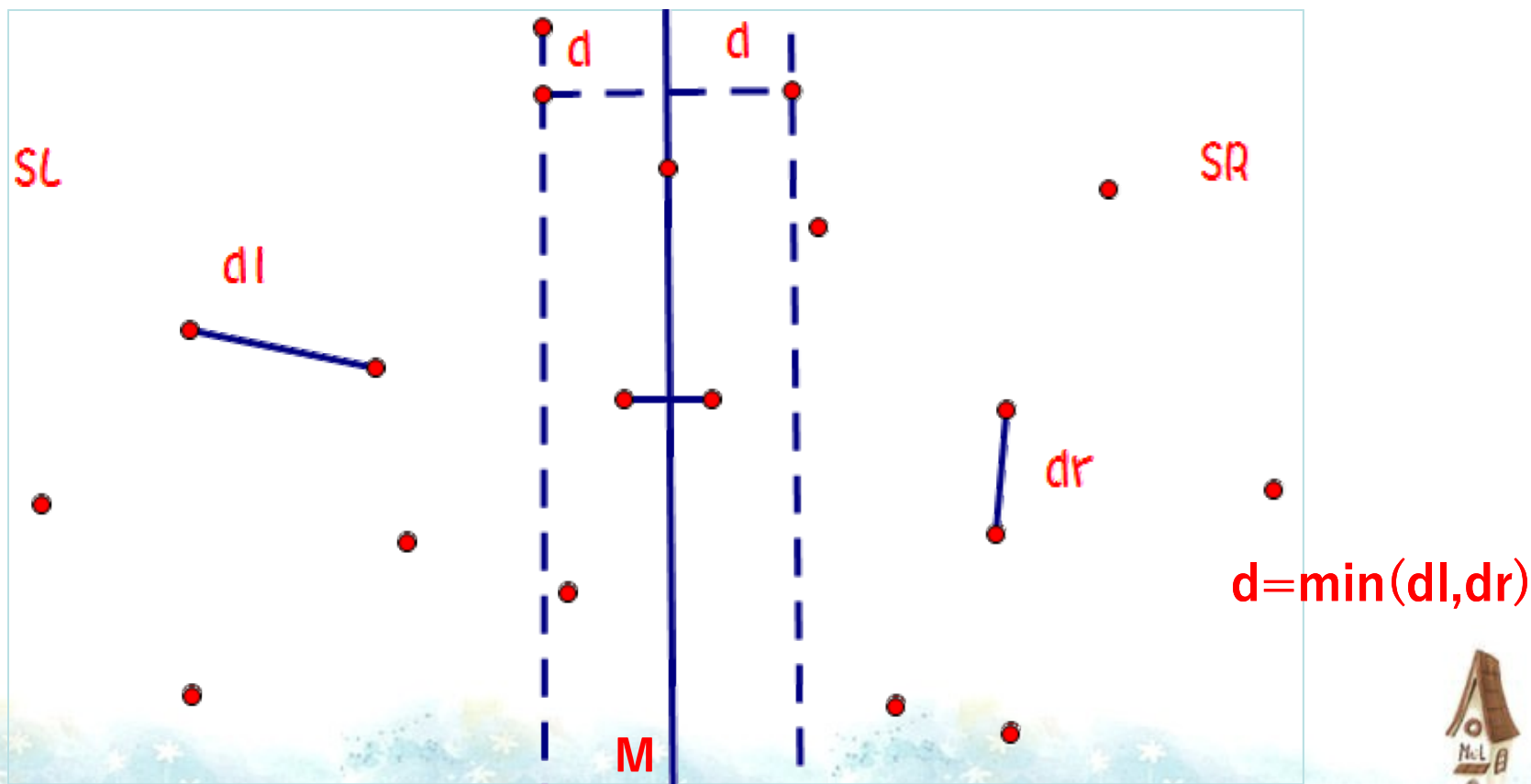


【算法分析】

$d = \min(d_l, d_r)$, 记录了SL, SR中的最短距离。

但这是不够的, S的最近点对的两个点很可能分别存在于SL, SR中。这种情况就要特殊处理了。

只有在下图中两虚线之间的点才有再计算的意义, 因为在这两条线之外的点与另一方的点的距离必然大于 d 。



【算法分析】

特殊讨论虚框中的点：

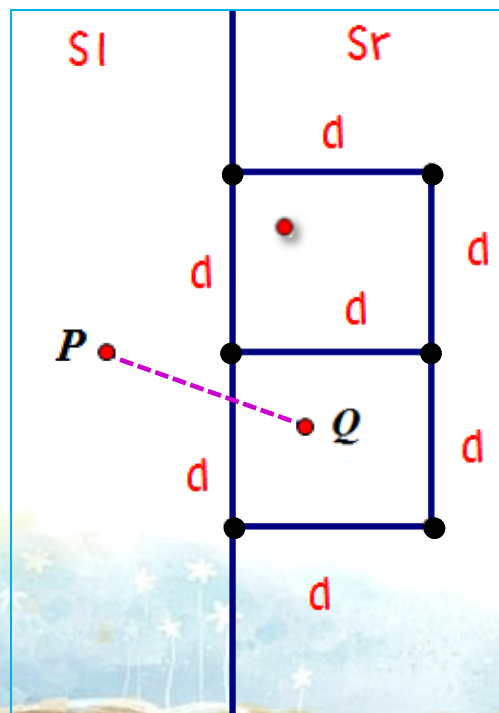
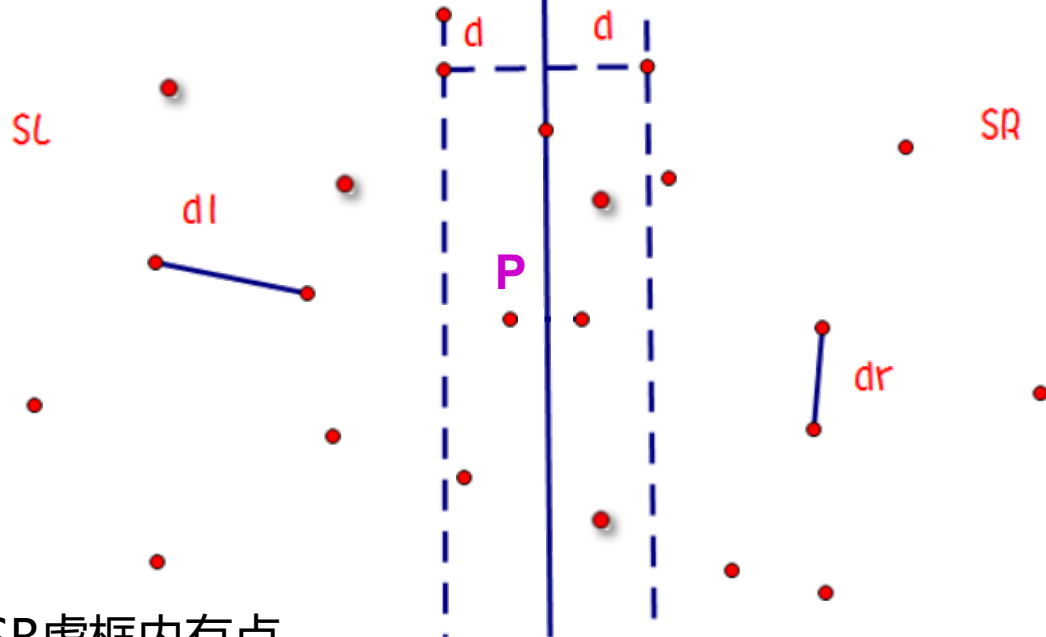
对于SL虚框范围内的某点P，我们可以依次讨论它到SR虚框中每个点的距离，但是SR虚框中的点可以很多，这样时间复杂度是 $O(n)$ 。

如果我们依次讨论SL虚框中每个点到SR虚框中每个点的距离，时间复杂度将是 $O(n^2)$ 。显然不可取。

对于SL虚框内的任意一点P，若在SR虚框内有点Q与它的距离小于d，那么Q点一定在如右图所示的两个正方形范围内。

观察：因为SR虚框中任意两点的距离都不小于d，所以我们知道在这两个正方形中，满足条件的点最多只有6个。（最多6个点就是图中的6个黑点，可由“鸽笼原理”证明）

所以，对于SL虚框内的任意一点P，我们只需要讨论SR虚框内与其y坐标之差不超过d的点，而这些点最多只有6个。



【算法分析】

对于SL虚框中任意一点P，怎样快速找出SR虚框中与P点距离不超过d的点？

- 1.将SL虚框中的点和SR虚框中的点全部放入集合T中，设总共有K个点。
- 2.对T中的点按y坐标由小到大排序。
- 3.按y坐标从小到大依次讨论T中的每个点。

对于i号点，讨论它与i+1号点的距离、与i+2号点的距离.....直到与i+x号点的距离大于d，便停止对i的讨论。同时记录下讨论中得到的最小的距离。

根据前面的分析，每个点最多讨论与其他6个点的距离，因此总讨论次数不超过6K。



```

double find(int l,int r)                                //找出区间[l,r]中的最近点对的距离,即编号l到r号点构成的区间
{
    if(r==l+1)return dis(l,r);                          //自定义函数dis(a,b)作用是计算a号点到b号点的欧几里得距离
    if(l+2==r)return min(dis(l,r), min(dis(l,l+1), dis(l+1,r)));

    int mid=(l+r)/2;
    double d=min( find(l,mid) , find(mid+1,r) );        //分别讨论[l,mid]和[mid+1,r]区间
    //上面代码表示的就是前面讲义中的d=min(dl,dr);

    int i,j,cnt=0;
    for(i=l;i<=r;i++)
        if(x[i]>=x[mid]-d&& x[i]<=x[mid]+d)T[++cnt]=i;    //数组T记录下虚框中的所有点的编号

    T_sortY(1,cnt);                                      //对T数组按y坐标由小到大排序

    for(i=1;i<=cnt;i++)                                  //讨论T数组的每个点i,看看y坐标比i号点大的点中, 有没有距离小于d的
        for(j=i+1;j<=cnt;j++)
        {
            if(y[T[j]]-y[T[i]]>=d) break;                //若T[j]号点与T[i]号在y坐标的距离大于了d,则结束i
            d=min(d,dis(T[i],T[j]));                     //若T[j]号点与T[i]号的距离小于d,则更新d
        }
    return d;
}

```

时间复杂度不超过 $O(6n\log n)$



延伸思考：如果是求三维空间的最近点对，该怎么解决？

